Wiktor Sowański Łukasz Sollich 203587, 203228

Sprawozdanie z projektu z „Metod Modelowania Matematycznego”

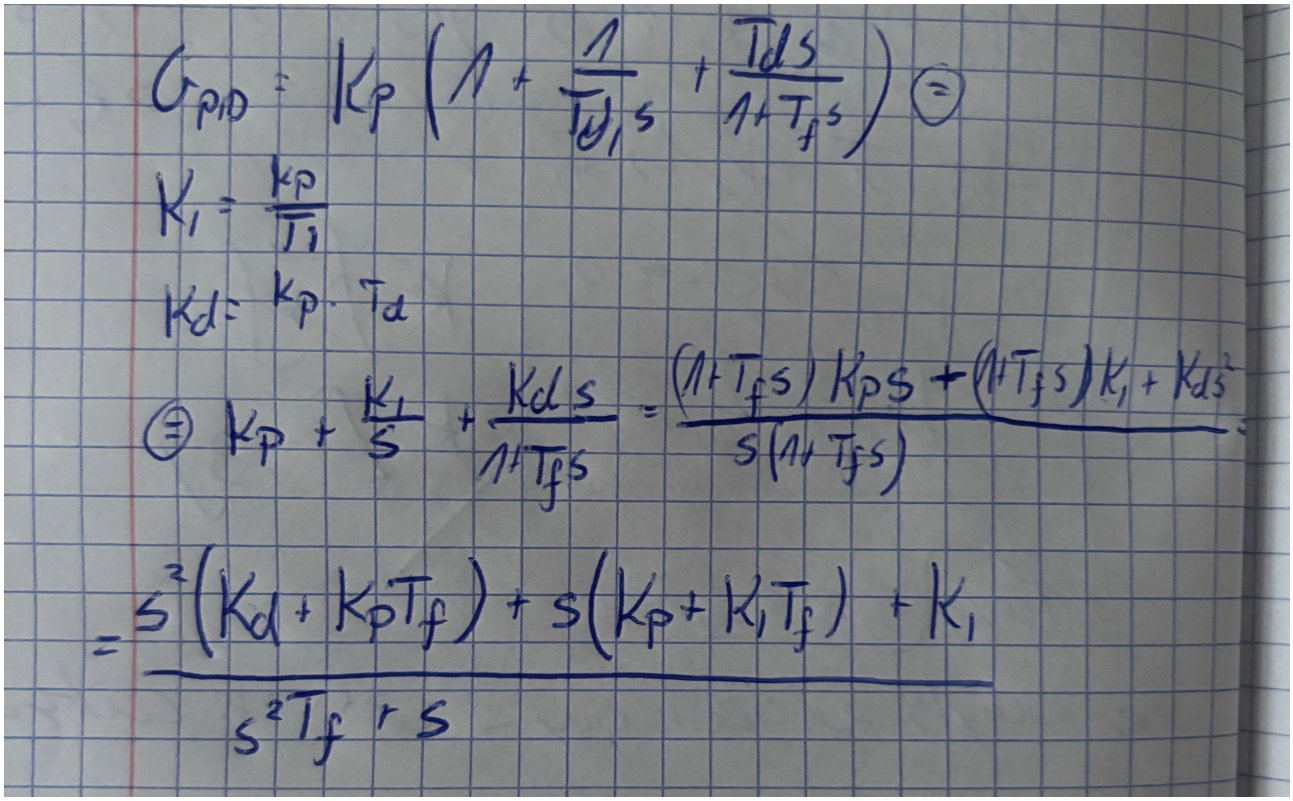
Celem projektu było utworzenie symulatora układu ze sprzężeniem zwrotnym oraz regulatorem PID na pobudzenie trzema rodzajami sygnałów. Jako sygnały w realizacji przyjęto sygnały: harmoniczny, piło kształtny oraz prostokątny.

1. Część Matematyczna:

A. **Regulator PID.** Regulator PID to obiekt posiadający transmitancję G = Kp*(1+ 1/(Ti*s) + Td*s) w zadaniu jednak sprecyzowano że regulator musi być możliwy do zrealizowania. Jako że idealny człon różniczkujący jest nie możliwy do zrealizowania transmitancja rzeczywistego regulatora PID przybiera postać G = Kp*(1+ 1/(Ti*s) + Td*s/(1+Tf*s)), gdzie Tf jest 5-20 razy mniejsza niż Td. W programie przyjęto wartość Tf = 0.01. Transmitancja takiego regulatora wynosi G = (s^2*(Kd+Kp*Tf) + s*(Kp+Ki*Tf) + Ki)/(s^2*Tf+s), obliczenia na rysunku poniżej.

B. **Obiekt.** Transmitancja obiektu wynosi G = (a1*s + a0)/(b2*s^2+b1*s+b0). Z twierdzenia Masona mamy H = Gpid*Gobiektu/(1+ Gpid*Gobiektu). Program następnie wykonuje symulację numeryczną z pomocą modelu stanowego układu wykorzystując funkcję „scipy.signal.lsim” w ten sposób generując przebieg dla wybranego wcześniej sygnału.

C. **Testy**. Symulacja została przetestowana na otwartej pętli regulatora (Parametry regulatora ustawione na 0, człon proporcjonalny ustawiony na 1). Otrzymano wynik dążący do stałej wartości dla biegunów o wartościach leżących w lewej półpłaszczyźnie i dąży do nieskończoności dla biegunów lezących na prawej półpłaszczyźnie, obiekt zachowuje się więc zgodnie z przewidywaniami.



2. Część Programistyczna:

A. **Biblioteki:** Przy realizacji projektu wykorzystano następujące biblioteki:
- Tkinter - do oprawy graficznej
- Numpy - do obliczeń matematycznych
- Scipy - do generowania i oprawy sygnałów oraz do symulacji
- Matplotlib - do rysowania wykresów sygnału i odpowiedzi

B. Interfejs Graficzny: Interfejs graficzny został zaprogramowany z wykorzystaniem biblioteki Tkinter pozwalającej na modularne budowanie oprawy graficznej. Po lewej stronie GUI umieszczono miejsce do wprowadzania parametrów symulacji oraz wyboru sygnału zadanego. Do dyspozycji użytkownik ma trzy typy sygnałów: harmoniczny, prostokątny i piło-kształtny. Po wybraniu typu sygnału program pokazuje miejsce do wprowadzenia parametrów wybranego sygnału. Po prawej stronie znajduje się wykres odpowiedzi układu oraz sygnału zadanego, dla pokazania wpływu obiektu oraz regulatora.

C. Sygnały: Program w zależności od typu sygnału prosi o odpowiednie dla sygnału parametry (np. Poziom wypełnienia dla prostokąta). W zależności od typu sygnału program generuje sygnał zgodnie z wymaganymi parametrami dla sygnału, rysunek poniżej.

```
        if sig_type == "Harmoniczny":
            amp = float(signal_frame.entries["amp"].get())
            omega = float(signal_frame.entries["omega"].get())
            phi = float(signal_frame.entries["phi"].get())
            u = amp * np.sin(omega * t + phi)

        elif sig_type == "Prostokąt":
            amp = float(signal_frame.entries["amp"].get())
            freq = float(signal_frame.entries["freq"].get())
            duty = float(signal_frame.entries["duty"].get())
            u = amp * (signal.square(2 * np.pi * freq * t, duty=duty) + 1) / 2

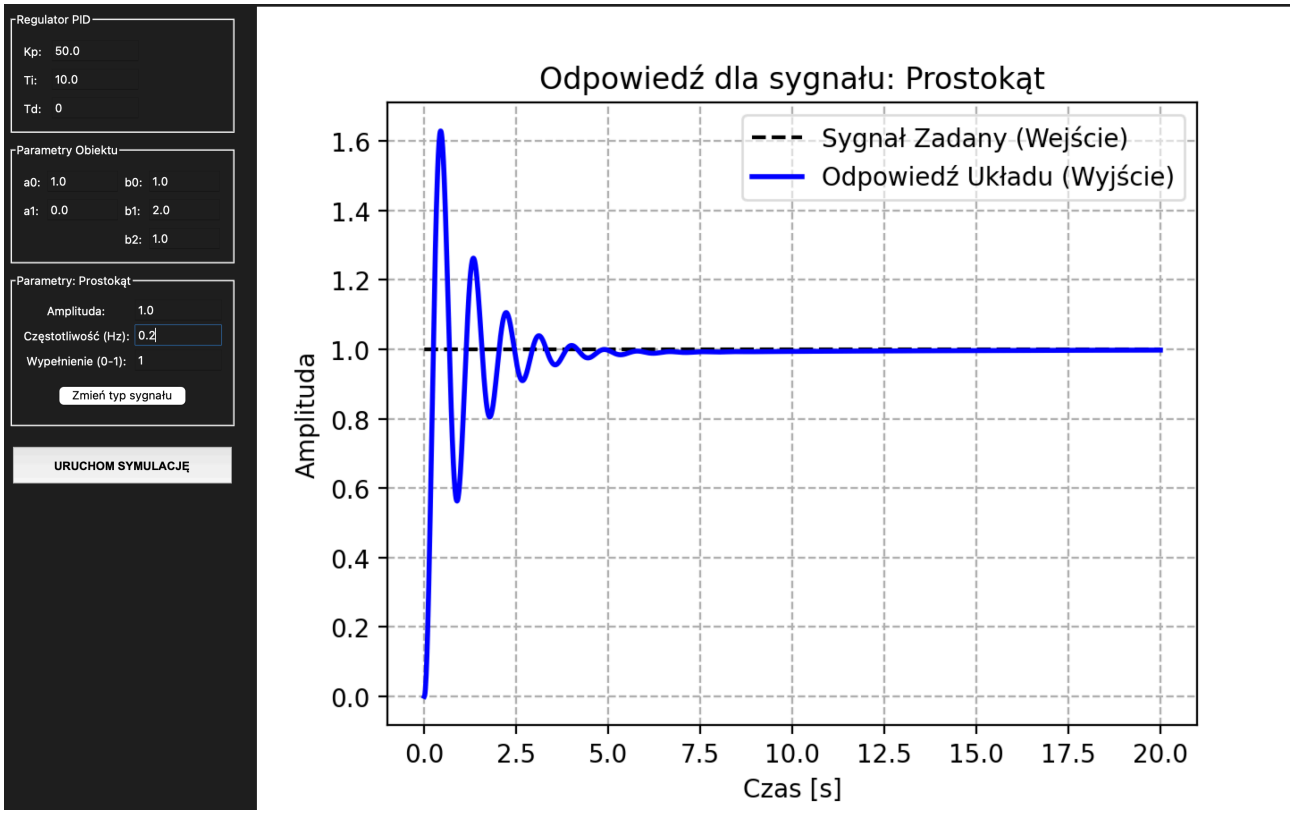
        elif sig_type == "Piłokształtny":
            amp = float(signal_frame.entries["amp"].get())
            freq = float(signal_frame.entries["freq"].get())
            width = float(signal_frame.entries["width"].get())
            u = amp * signal.sawtooth(2 * np.pi * freq * t, width=width)
```

D. Symulacja: Program wykonuje symulacje przy wykorzystaniu funkcji signal.lsim() z biblioteki Scipy. Program bierze transmitancje obiektu oraz regulatora, wektor czasu z biblioteki Numpy i na jej podstawie generuje odpowiedź układu. Symulacja jest opisana w funkcji run_simulation() przy wykorzystaniu mechanizmu try: except: dla zabezpieczenia przed brakiem wprowadzenia parametrów lub pozostałych wypadkach, rysunek poniżej. Funkcja signal.lsim() pobiera model stanowy układu, utworzony przy wykorzystaniu funkcji signal.Transferfunction() z transmitancji, sygnał wejściowy oraz czas symulacji. Wykorzystuje ona wewnętrzny numeryczny solver równań różniczkowych.

```
    except ValueError:
        messagebox.showerror("Błąd danych", "Upewnij się, że wszystkie pola zawierają poprawne liczby (używaj kropek,
    except Exception as e:
        messagebox.showerror("Błąd symulacji", f"Wystąpił nieoczekiwany błąd:\n{str(e)}")
```

E. **Rysowanie wykresu:** Z wykorzystaniem funkcji z biblioteki matplotlib narysowano wykresy sygnału zadanego dla wybranych parametrów oraz odpowiedzi układu

Poniżej przedstawiono przykładową pracę programu dla skoku jednostkowego:



3. Wnioski:

Układ generuje odpowiedź na skok jednostkowy zgodnie z przewidywaniami. Układ jest stabilny więc odpowiedź dąży do wartości ustalonej. Wykresy odpowiedzi pokrywają się ze znanymi z zajęć automatyki kształtami, dla różnych pobudzeń można zaobserwować przeregulowanie lub przy wybraniu biegunów na prawej półpłaszczyźnie można zaobserwować niestabilność układu co również zgadza się z teorią. Sprawdzenie poprawności modelu inną metodą jest niemożliwe z powodu braku dostępności mechanicznych bądź elektrycznych układów do symulowania oraz aparatury do mierzenia odpowiedzi rzeczywistych układów.

4. Źródła:

- „Podstawy Automatyki” tom 1. J. Nowakowski; Politechnika Gdańska 1992
- Podręcznik biblioteki „Scipy”
- Dokumentacja ze strony „python.org"